ZONE 4: REVIEW - API ENDPOINTS SPECIFICATION

---

3. API ENDPOINTS

Base URL: `/api/v1/review`

3.1 AFTER-ACTION REPORTS

Generate After-Action Report

typescript

```
TypeScript
POST /api/v1/review/reports/generate
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  event_id: string;
  report_type: 'automatic' | 'comprehensive' | 'executive' |
'compliance';
  include_recommendations?: boolean;
  include_benchmarks?: boolean;
  include_evidence_ledger?: boolean;
  distribution_list?: string[]; // User IDs or email addresses
  schedule_distribution?: {
    send_at?: Date;
    channels?: ('email' | 'dashboard' | 'api' | 'pdf')[];
  };
}
```

```
Response: 201 Created
{
  success: true;
  data: {
    report_id: string;
    status: 'generating';
    estimated_completion_seconds: number;
    tracking_url: string;
  };
  metadata: {
    request_id: string;
    timestamp: string;
  };
}

Response: 400 Bad Request
{
  success: false;
  error: {
    code: 'INVALID_EVENT';
    message: 'Event must be completed before generating report';
    details: {
      event_status: string;
      event_end_time: string;
    };
  };
}
```

Get Report Status

typescript

TypeScript

```
GET /api/v1/review/reports/{report_id}/status
Authorization: Bearer {token}
```

```
Response: 200 OK
{
  success: true;
  data: {
    report_id: string;
    status: 'generating' | 'review' | 'approved' | 'published';
    progress_percentage: number; // 0-100
    current_stage: string; // 'data_collection', 'analysis',
'report_generation', etc.
    estimated_completion: string; // ISO timestamp
    errors?: string[];
  };
}
```

Get After-Action Report

typescript

TypeScript

```
GET /api/v1/review/reports/{report_id}
Authorization: Bearer {token}
Query Parameters:
  - format?: 'json' | 'pdf' | 'html'
  - include_evidence?: boolean
  - include_raw_data?: boolean

Response: 200 OK (format=json)
{
  success: true;
  data: AfterActionReport; // Full report object
  metadata: {
    generated_at: string;
    report_version: string;
    data_sources: string[];
  };
}
```

```
Response: 200 OK (format=pdf)
Content-Type: application/pdf
Content-Disposition: attachment;
filename="AAR-{venue}-{event}-{date}.pdf"
[Binary PDF data]
```

List Reports

typescript

TypeScript

```
GET /api/v1/review/reports
Authorization: Bearer {token}
Query Parameters:
  - venue_id?: string
  - event_id?: string
  - report_type?: string
  - status?: string
  - start_date?: string (ISO date)
  - end_date?: string (ISO date)
  - page?: number (default: 1)
  - limit?: number (default: 20, max: 100)
  - sort_by?: 'event_date' | 'created_at' | 'report_status'
  - sort_order?: 'asc' | 'desc'

Response: 200 OK
{
  success: true;
  data: {
    reports: AfterActionReport[];
    pagination: {
      current_page: number;
      total_pages: number;
      total_count: number;
      has_next: boolean;
```

```
      has_prev: boolean;
    };
  };
}
```

Approve Report

typescript

```
TypeScript
POST /api/v1/review/reports/{report_id}/approve
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  approval_notes?: string;
  publish_immediately?: boolean;
  distribution_override?: string[];
}

Response: 200 OK
{
  success: true;
  data: {
    report_id: string;
    status: 'approved';
    approved_by: string;
    approved_at: string;
    published: boolean;
  };
}
```

Publish Report

typescript

```
TypeScript
POST /api/v1/review/reports/{report_id}/publish
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  distribution_channels: ('email' | 'dashboard' | 'api' |
'pdf')[];
  recipients: string[];
  notification_message?: string;
}

Response: 200 OK
{
  success: true;
  data: {
    report_id: string;
    published_at: string;
    distribution_summary: {
      total_recipients: number;
      channels_used: string[];
      delivery_status: {
        delivered: number;
        pending: number;
        failed: number;
      };
    };
  };
}
```

3.2 PERFORMANCE METRICS & ANALYTICS

Get Event Performance Metrics

typescript

```typescript
GET /api/v1/review/performance/events/{event_id}
Authorization: Bearer {token}
Query Parameters:
  - include_benchmarks?: boolean
  - include_historical?: boolean
  - compare_to_event_id?: string

Response: 200 OK
{
  success: true;
  data: {
    event_metrics: EventPerformanceMetrics;
    benchmarks?: {
      league_comparison: ComparisonData[];
      historical_comparison: ComparisonData[];
      peer_comparison: ComparisonData[];
    };
    insights: {
      key_achievements: string[];
      improvement_areas: string[];
      notable_changes: string[];
    };
  };
}
```

Analyze Performance Trends

typescript

```typescript
POST /api/v1/review/performance/trends
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
```

```
  venue_id: string;
  start_date: string;
  end_date: string;
  metrics: string[]; // Metric names to analyze
  aggregation?: 'daily' | 'weekly' | 'monthly';
  include_predictions?: boolean;
}

Response: 200 OK
{
  success: true;
  data: {
    trends: {
      [metric_name: string]: {
        dates: string[];
        values: number[];
        trend_direction: 'improving' | 'stable' | 'declining';
        change_percentage: number;
        statistical_significance: number;
        predictions?: {
          next_period_value: number;
          confidence_interval: [number, number];
          forecast_accuracy: number;
        };
      };
    };
    summary: {
      improving_metrics: string[];
      declining_metrics: string[];
      stable_metrics: string[];
      overall_trend: string;
    };
  };
}
```

Compare Venue Performance

typescript

```
TypeScript
POST /api/v1/review/performance/compare
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_ids: string[];
  period_start: string;
  period_end: string;
  metrics: string[];
  normalize?: boolean; // Normalize for venue size/capacity
}

Response: 200 OK
{
  success: true;
  data: {
    comparison: {
      [metric_name: string]: {
        venues: {
          venue_id: string;
          venue_name: string;
          value: number;
          rank: number;
        }[];
        league_average: number;
        best_practice_value: number;
      };
    };
    insights: {
      top_performers: { venue_id: string; metric: string; value:
number }[];
      improvement_opportunities: { venue_id: string; metric:
string; gap: number }[];
    };
```

```
  };
}
```

Get Performance Dashboard

typescript

```
TypeScript
GET /api/v1/review/performance/dashboard/{venue_id}
Authorization: Bearer {token}
Query Parameters:
  - period?: 'current_month' | 'last_30_days' | 'current_quarter'
| 'custom'
  - start_date?: string (required if period=custom)
  - end_date?: string (required if period=custom)

Response: 200 OK
{
  success: true;
  data: {
    summary: {
      overall_score: number; // 0-100
      total_events: number;
      total_incidents: number;
      incidents_prevented: number;
      avg_response_time_seconds: number;
      guest_satisfaction_avg: number;
      compliance_score: number;
    };
    category_scores: {
      safety: number;
      operations: number;
      efficiency: number;
      satisfaction: number;
    };
    trends: {
```

```
      [metric_name: string]: {
        current_value: number;
        previous_value: number;
        change_pct: number;
        sparkline_data: number[]; // Last 10 data points
      };
    };
    alerts: {
      declining_metrics: string[];
      improvement_opportunities: string[];
      notable_achievements: string[];
    };
  };
}
```

3.3 INCIDENT ANALYSIS

Analyze Incident

typescript

TypeScript

```
POST /api/v1/review/incidents/{incident_id}/analyze
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  analysis_type: 'root_cause' | 'response_effectiveness' |
'prevention' | 'comprehensive';
  include_similar_incidents?: boolean;
  generate_recommendations?: boolean;
}

Response: 201 Created
```

```
{
  success: true;
  data: {
    analysis_id: string;
    status: 'analyzing';
    estimated_completion_seconds: number;
  };
}
```

Get Incident Analysis

typescript

```
TypeScript
GET /api/v1/review/incidents/{incident_id}/analysis
Authorization: Bearer {token}
Query Parameters:
  - analysis_id?: string (get specific analysis)
  - include_evidence?: boolean

Response: 200 OK
{
  success: true;
  data: IncidentAnalysis;
}
```

Bulk Analyze Incidents

typescript

```
TypeScript
POST /api/v1/review/incidents/analyze-bulk
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
```

```
  incident_ids?: string[]; // Specific incidents
  event_id?: string; // All incidents from event
  venue_id?: string; // Requires date range
  start_date?: string;
  end_date?: string;
  filters?: {
    severity?: string[];
    category?: string[];
    preventability?: string[];
  };
}

Response: 202 Accepted
{
  success: true;
  data: {
    job_id: string;
    total_incidents: number;
    estimated_completion_minutes: number;
    status_url: string;
  };
}
```

Get Pattern Analysis

typescript

TypeScript

```
GET /api/v1/review/patterns
Authorization: Bearer {token}
Query Parameters:
  - venue_id: string (required)
  - pattern_type?: string
  - confidence_threshold?: number (0-100, default: 70)
  - start_date?: string
  - end_date?: string
```

```
  - limit?: number

Response: 200 OK
{
  success: true;
  data: {
    patterns: PatternAnalysis[];
    summary: {
      total_patterns: number;
      critical_patterns: number;
      high_confidence_patterns: number;
      recommended_actions: string[];
    };
  };
}
```

Detect New Patterns

typescript

```
TypeScript
POST /api/v1/review/patterns/detect
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  start_date: string;
  end_date: string;
  pattern_types?: string[];
  confidence_threshold?: number;
  ml_enabled?: boolean;
}

Response: 202 Accepted
```

```
{
  success: true;
  data: {
    job_id: string;
    analysis_scope: {
      events_analyzed: number;
      incidents_analyzed: number;
      date_range: { start: string; end: string };
    };
    estimated_completion_minutes: number;
  };
}
```

3.4 IMPROVEMENT RECOMMENDATIONS

Create Recommendation

typescript

TypeScript

```
POST /api/v1/review/recommendations
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  recommendation_title: string;
  recommendation_description: string;
  recommendation_type: 'process' | 'technology' | 'training' |
'policy' | 'infrastructure';
  priority_level: 'critical' | 'high' | 'medium' | 'low';
  urgency: 'immediate' | 'short_term' | 'medium_term' |
'long_term';
  estimated_impact: 'high' | 'medium' | 'low';
```

```
  source_incident_id?: string;
  source_analysis_id?: string;
  estimated_cost?: number;
  estimated_hours?: number;
  required_resources?: string[];
  success_criteria?: string[];
}

Response: 201 Created
{
  success: true;
  data: ImprovementRecommendation;
}
```

List Recommendations

typescript

```
TypeScript
GET /api/v1/review/recommendations
Authorization: Bearer {token}
Query Parameters:
  - venue_id?: string
  - status?: string
  - priority?: string
  - type?: string
  - assigned_to?: string
  - page?: number
  - limit?: number
  - sort_by?: 'priority' | 'created_at' | 'target_date'
  - sort_order?: 'asc' | 'desc'

Response: 200 OK
{
  success: true;
  data: {
```

```
    recommendations: ImprovementRecommendation[];
    summary: {
      total: number;
      by_status: { [status: string]: number };
      by_priority: { [priority: string]: number };
      overdue: number;
    };
    pagination: PaginationMetadata;
  };
}
```

Update Recommendation Status

typescript

```
TypeScript
PATCH /api/v1/review/recommendations/{recommendation_id}/status
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  status: 'proposed' | 'approved' | 'in_progress' | 'completed' |
'rejected' | 'deferred';
  notes?: string;
  implementation_owner?: string;
  target_date?: string;
}

Response: 200 OK
{
  success: true;
  data: ImprovementRecommendation;
}
```

Approve Recommendation

typescript

```
TypeScript
POST /api/v1/review/recommendations/{recommendation_id}/approve
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  approval_notes?: string;
  assigned_to?: string;
  budget_allocated?: number;
  target_completion_date?: string;
  priority_override?: 'critical' | 'high' | 'medium' | 'low';
}

Response: 200 OK
{
  success: true;
  data: ImprovementRecommendation;
}
```

Get Recommendation ROI Analysis

typescript

```
TypeScript
GET /api/v1/review/recommendations/{recommendation_id}/roi
Authorization: Bearer {token}

Response: 200 OK
{
  success: true;
  data: {
    recommendation_id: string;
    estimated_costs: {
      implementation: number;
      training: number;
      ongoing_maintenance: number;
```

```
      total: number;
    };
    estimated_benefits: {
      cost_savings_annual: number;
      efficiency_gains_hours: number;
      risk_reduction_value: number;
      revenue_impact: number;
      total_annual: number;
    };
    roi_analysis: {
      payback_period_months: number;
      roi_percentage: number;
      five_year_npv: number;
      break_even_date: string;
    };
    confidence_level: 'high' | 'medium' | 'low';
    assumptions: string[];
  };
}
```

3.5 IMPLEMENTATION TRACKING

Create Implementation

typescript

TypeScript

```
POST /api/v1/review/implementations
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  recommendation_id: string;
  implementation_plan: string;
```

```
  implementation_approach: 'phased' | 'immediate' | 'pilot' |
'full_rollout';
  project_lead: string;
  team_members: string[];
  budget_allocated: number;
  hours_estimated: number;
  milestones: Milestone[];
  rollback_plan?: string;
}

Response: 201 Created
{
  success: true;
  data: ImprovementImplementation;
}
```

Update Implementation Progress

typescript

```
TypeScript
PATCH /api/v1/review/implementations/{implementation_id}/progress
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
  progress_status: 'planning' | 'executing' | 'testing' |
'validating' | 'completed';
  percent_complete: number; // 0-100
  current_milestone?: string;
  budget_spent?: number;
  hours_spent?: number;
  progress_notes?: string;
  blockers?: string[];
}
```

```
Response: 200 OK
{
  success: true;
  data: ImprovementImplementation;
}
```

Complete Milestone

typescript

TypeScript

```
POST
/api/v1/review/implementations/{implementation_id}/milestones/{mi
lestone_id}/complete
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  completion_notes?: string;
  actual_completion_date?: string;
  evidence_refs?: string[];
}

Response: 200 OK
{
  success: true;
  data: {
    milestone: Milestone;
    implementation_progress: number; // Overall % complete
    next_milestone?: Milestone;
  };
}
```

Run Validation Tests

typescript

```
TypeScript
POST /api/v1/review/implementations/{implementation_id}/validate
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  validation_tests: {
    test_name: string;
    test_description: string;
    test_procedures: string[];
    success_criteria: string[];
  }[];
  schedule_date?: string; // If not immediate
}

Response: 202 Accepted
{
  success: true;
  data: {
    validation_id: string;
    scheduled_date: string;
    total_tests: number;
    estimated_duration_minutes: number;
  };
}
```

Get Implementation Dashboard

typescript

```
TypeScript
GET /api/v1/review/implementations/dashboard/{venue_id}
Authorization: Bearer {token}
Query Parameters:
  - status?: string
  - priority?: string
```

```
Response: 200 OK
{
  success: true;
  data: {
    summary: {
      total_implementations: number;
      in_progress: number;
      completed: number;
      overdue: number;
      total_budget_allocated: number;
      total_budget_spent: number;
      total_roi_realized: number;
    };
    active_implementations: ImprovementImplementation[];
    upcoming_milestones: Milestone[];
    overdue_items: {
      implementations: ImprovementImplementation[];
      milestones: Milestone[];
    };
    recent_completions: ImprovementImplementation[];
  };
}
```

3.6 BENCHMARKING & COMPARISON

Get Benchmarks

typescript

TypeScript

```
GET /api/v1/review/benchmarks
Authorization: Bearer {token}
Query Parameters:
  - venue_id: string (required)
```

```
  - benchmark_type: 'league' | 'division' | 'conference' |
'venue_tier' | 'custom'
  - metric_category?: string
  - metric_name?: string
  - period_start: string (ISO date)
  - period_end: string (ISO date)

Response: 200 OK
{
  success: true;
  data: {
    benchmarks: BenchmarkMetric[];
    venue_summary: {
      overall_percentile: number;
      top_performing_metrics: string[];
      improvement_needed_metrics: string[];
      competitive_position: 'leader' | 'above_average' |
'average' | 'below_average';
    };
  };
}
```

Generate Benchmark Report

typescript

TypeScript

```
POST /api/v1/review/benchmarks/report
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  benchmark_type: 'league' | 'division' | 'peer_group';
  period_start: string;
```

```
  period_end: string;
  metrics: string[]; // Specific metrics or 'all'
  include_trends?: boolean;
  include_recommendations?: boolean;
  format?: 'json' | 'pdf';
}

Response: 200 OK
{
  success: true;
  data: {
    report_id: string;
    venue_performance: {
      overall_ranking: number;
      total_venues: number;
      percentile: number;
      category_rankings: {
        [category: string]: {
          rank: number;
          score: number;
          percentile: number;
        };
      };
    };
    comparisons: BenchmarkComparison[];
    trends: {
      [metric: string]: {
        current_period: number;
        previous_period: number;
        change_pct: number;
        relative_to_league: 'improving_faster' |
'improving_slower' | 'declining';
      };
    };
    recommendations?: string[];
  };
```

```
}
```

Compare to Peer Venues

typescript

```
TypeScript
POST /api/v1/review/benchmarks/peer-comparison
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  peer_venue_ids?: string[]; // If not provided, system selects
similar venues
  metrics: string[];
  period_start: string;
  period_end: string;
  normalization?: 'capacity' | 'revenue' | 'none';
}

Response: 200 OK
{
  success: true;
  data: {
    venue_performance: { [metric: string]: number };
    peer_averages: { [metric: string]: number };
    detailed_comparison: {
      [metric: string]: {
        venue_value: number;
        venue_rank: number;
        peer_values: {
          venue_id: string;
          venue_name: string;
          value: number;
```

```
        }[];
        best_practice_venue: string;
        best_practice_value: number;
      };
    };
    insights: {
      strengths: string[];
      opportunities: string[];
      learning_opportunities: {
        metric: string;
        best_practice_venue: string;
        potential_improvement: number;
      }[];
    };
  };
}
```

3.7 BEST PRACTICES & KNOWLEDGE SHARING

Get Best Practices

typescript

TypeScript

```
GET /api/v1/review/best-practices
Authorization: Bearer {token}
Query Parameters:
  - category?: string
  - validation_status?: string
  - implementation_difficulty?: string
  - min_adoption_count?: number
  - sort_by?: 'adoption_count' | 'roi' | 'validation_status'

Response: 200 OK
{
```

```
  success: true;
  data: {
    best_practices: BestPractice[];
    summary: {
      total_practices: number;
      by_category: { [category: string]: number };
      by_validation: { [status: string]: number };
      most_adopted: BestPractice[];
      highest_roi: BestPractice[];
    };
  };
}
```

Submit Best Practice

typescript

TypeScript

```
POST /api/v1/review/best-practices
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  practice_title: string;
  practice_description: string;
  practice_category: string;
  originated_from_venue_id?: string;
  originated_from_event_id?: string;
  implementation_difficulty: 'easy' | 'moderate' | 'complex';
  estimated_implementation_time: string;
  required_resources: string[];
  implementation_steps: any;
  impact_metrics?: {
    cost_savings?: number;
    efficiency_gain_pct?: number;
```

```
    satisfaction_improvement?: number;
    safety_improvement?: string;
  };
  documentation_links?: string[];
}

Response: 201 Created
{
  success: true;
  data: BestPractice;
}
```

Adopt Best Practice

typescript

TypeScript

```
POST /api/v1/review/best-practices/{practice_id}/adopt
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  implementation_plan?: string;
  target_completion_date?: string;
  assigned_to?: string;
}

Response: 200 OK
{
  success: true;
  data: {
    adoption_id: string;
    practice: BestPractice;
    implementation_created: boolean;
```

```
    implementation_id?: string;
  };
}
```

Report Best Practice Results

typescript

TypeScript

```
POST /api/v1/review/best-practices/{practice_id}/results
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  success: boolean;
  actual_cost_savings?: number;
  actual_efficiency_gain?: number;
  actual_satisfaction_improvement?: number;
  lessons_learned: string[];
  implementation_notes: string;
  would_recommend: boolean;
}

Response: 200 OK
{
  success: true;
  data: {
    practice_id: string;
    validation_updated: boolean;
    new_validation_success_rate: number;
  };
}
```

3.8 EVIDENCE LEDGER INTEGRATION

Query Evidence Ledger

typescript

```
TypeScript
POST /api/v1/review/evidence/query
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  query_type: 'incident_timeline' | 'decision_audit' |
'compliance_evidence' | 'performance_data';
  venue_id: string;
  event_id?: string;
  incident_id?: string;
  start_time?: string;
  end_time?: string;
  filters?: {
    action_types?: string[];
    user_ids?: string[];
    systems?: string[];
  };
  include_metadata?: boolean;
}

Response: 200 OK
{
  success: true;
  data: {
    query_id: string;
    ledger_transactions: {
      transaction_id: string;
      timestamp: string;
      action_type: string;
      actor: string;
      system: string;
```

```
      data_hash: string;
      metadata?: any;
    }[];
    summary: {
      total_records: number;
      time_span: { start: string; end: string };
      data_integrity_verified: boolean;
    };
  };
}
```

Verify Evidence Chain

typescript

TypeScript

```
POST /api/v1/review/evidence/verify
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  transaction_ids: string[];
  verification_type: 'integrity' | 'completeness' |
'chain_of_custody';
}

Response: 200 OK
{
  success: true;
  data: {
    verification_result: 'verified' | 'failed' | 'incomplete';
    verified_transactions: number;
    failed_transactions: number;
    integrity_score: number; // 0-100
    issues?: {
```

```
      transaction_id: string;
      issue_type: string;
      description: string;
    }[];
  };
}
```

Export Evidence Package

typescript

TypeScript

```
POST /api/v1/review/evidence/export
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  event_id?: string;
  incident_id?: string;
  report_id?: string;
  start_time?: string;
  end_time?: string;
  format: 'json' | 'pdf' | 'blockchain_export';
  include_media?: boolean;
  certification_required?: boolean;
}

Response: 202 Accepted
{
  success: true;
  data: {
    export_id: string;
    estimated_completion_minutes: number;
    download_url_available_at: string; // URL to check status
```

```
  };
}
```

3.9 CONTINUOUS IMPROVEMENT WORKFLOWS

Create Workflow

typescript

```
TypeScript
POST /api/v1/review/workflows
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  workflow_name: string;
  workflow_type: 'incident_review' | 'quarterly_review' |
'compliance_audit' | 'performance_review';
  trigger_type?: 'event_based' | 'scheduled' | 'manual' |
'threshold';
  trigger_condition?: string;
  recurrence_pattern?: string;
  workflow_owner: string;
  participants: string[];
  approvers: string[];
  stages: WorkflowStage[];
  due_date?: string;
}

Response: 201 Created
{
  success: true;
  data: ImprovementWorkflow;
```

```
}
```

Start Workflow

typescript

TypeScript

```
POST /api/v1/review/workflows/{workflow_id}/start
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  context?: {
    event_id?: string;
    report_id?: string;
    incident_ids?: string[];
  };
  priority_override?: 'high' | 'normal' | 'low';
}

Response: 200 OK
{
  success: true;
  data: {
    workflow_id: string;
    started_at: string;
    current_stage: string;
    next_actions: string[];
    assigned_participants: string[];
  };
}
```

Update Workflow Stage

typescript

TypeScript

```
PATCH /api/v1/review/workflows/{workflow_id}/stages/{stage_id}
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  status: 'pending' | 'in_progress' | 'completed' | 'skipped';
  notes?: string;
  completed_by?: string;
  artifacts?: string[]; // URLs or IDs of generated documents
}

Response: 200 OK
{
  success: true;
  data: {
    workflow: ImprovementWorkflow;
    stage_updated: WorkflowStage;
    next_stage?: WorkflowStage;
    workflow_completed: boolean;
  };
}
```

Complete Workflow

typescript

TypeScript

```
POST /api/v1/review/workflows/{workflow_id}/complete
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  outcome_summary: string;
  action_items: string[];
```

```
  follow_up_required?: boolean;
  follow_up_date?: string;
  artifacts?: string[];
}

Response: 200 OK
{
  success: true;
  data: {
    workflow_id: string;
    completed_at: string;
    total_duration_hours: number;
    action_items_created: number;
    recommendations_generated: number;
    next_scheduled_occurrence?: string; // If recurring
  };
}
```

Get Active Workflows

typescript

TypeScript

```
GET /api/v1/review/workflows/active
Authorization: Bearer {token}
Query Parameters:
  - venue_id?: string
  - workflow_type?: string
  - assigned_to?: string
  - overdue_only?: boolean

Response: 200 OK
{
  success: true;
  data: {
    active_workflows: ImprovementWorkflow[];
```

```
    summary: {
      total_active: number;
      overdue: number;
      due_this_week: number;
      by_type: { [type: string]: number };
    };
  };
}
```

3.10 CONTINUOUS IMPROVEMENT DASHBOARD

Get Improvement Dashboard

typescript

```
TypeScript
GET /api/v1/review/dashboard/{venue_id}
Authorization: Bearer {token}
Query Parameters:
  - period?: 'current_month' | 'current_quarter' | 'current_year'
| 'custom'
  - start_date?: string
  - end_date?: string

Response: 200 OK
{
  success: true;
  data: ContinuousImprovementDashboard; // Full dashboard object
}
```

Get Improvement Insights

typescript

```
TypeScript
POST /api/v1/review/insights/generate
```

```
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  insight_types: ('performance_trends' | 'pattern_detection' |
'benchmark_comparison' | 'recommendation_priorities')[];
  period_start: string;
  period_end: string;
  ai_powered?: boolean;
}

Response: 200 OK
{
  success: true;
  data: {
    insights: {
      type: string;
      priority: 'critical' | 'high' | 'medium' | 'low';
      title: string;
      description: string;
      supporting_data: any;
      recommended_actions: string[];
      estimated_impact: {
        cost_savings?: number;
        efficiency_gain?: number;
        risk_reduction?: string;
      };
    }[];
    action_plan: {
      immediate_actions: string[];
      short_term_actions: string[];
      long_term_initiatives: string[];
    };
  };
```

```
}
```

3.11 INTEGRATION ENDPOINTS

Sync with PREPARE Zone

typescript

```TypeScript
POST /api/v1/review/sync/prepare
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  venue_id: string;
  recommendations: string[]; // Recommendation IDs to sync back to
blueprints
  best_practices: string[]; // Best practice IDs to integrate
  update_blueprints?: boolean;
  update_compliance_frameworks?: boolean;
}

Response: 200 OK
{
  success: true;
  data: {
    blueprints_updated: number;
    compliance_frameworks_updated: number;
    playbooks_enhanced: number;
    sync_summary: string;
  };
}
```

Export to External Systems

typescript

```typescript
POST /api/v1/review/export/external
Authorization: Bearer {token}
Content-Type: application/json

Request Body:
{
  export_type: 'after_action_report' | 'performance_metrics' |
'improvement_recommendations';
  target_system: 'email' | 'slack' | 'teams' | 'jira' |
'servicenow' | 'custom_webhook';
  config: {
    [key: string]: any; // System-specific configuration
  };
  report_id?: string;
  venue_id?: string;
  event_id?: string;
}

Response: 200 OK
{
  success: true;
  data: {
    export_id: string;
    target_system: string;
    delivery_status: 'sent' | 'queued' | 'failed';
    external_reference?: string; // External system's ID
  };
}
```

This completes the comprehensive API endpoint specification for ZONE 4: REVIEW.

Next sections:

- **Section 4: UI Components** (React components for all review interfaces)
- **Section 5: Temporal Sequence** (detailed T+0 to T+30 day timeline)
- **Section 6: State Machine** (REVIEW zone states and transitions)

Shall I continue with Section 4: UI Components?

continue with UI components. When you are about to run out of space provide prompt for next thread

12:51 PM

ZONE 4: REVIEW - UI COMPONENTS SPECIFICATION

---

4. UI COMPONENTS

Component Library Structure

```
None
src/components/review/
├── dashboards/
│   ├── AfterActionDashboard.tsx
│   ├── PerformanceDashboard.tsx
│   ├── ImprovementDashboard.tsx
│   └── ExecutiveSummary.tsx
├── reports/
│   ├── ReportGenerator.tsx
│   ├── ReportViewer.tsx
│   ├── ReportList.tsx
│   └── ReportApproval.tsx
├── analytics/
│   ├── PerformanceMetrics.tsx
│   ├── TrendAnalysis.tsx
│   ├── BenchmarkComparison.tsx
│   └── PatternDetection.tsx
├── incidents/
│   ├── IncidentAnalysis.tsx
│   ├── RootCauseAnalyzer.tsx
│   ├── LessonsLearned.tsx
```

```
|   └── SimilarIncidents.tsx
├── improvements/
|   ├── RecommendationList.tsx
|   ├── RecommendationCard.tsx
|   ├── ImplementationTracker.tsx
|   └── ROICalculator.tsx
├── evidence/
|   ├── EvidenceLedgerViewer.tsx
|   ├── EvidenceTimeline.tsx
|   └── EvidenceExporter.tsx
├── workflows/
|   ├── WorkflowBuilder.tsx
|   ├── WorkflowTracker.tsx
|   └── WorkflowStageCard.tsx
└── shared/
    ├── MetricCard.tsx
    ├── TrendChart.tsx
    ├── ComparisonTable.tsx
    └── InsightPanel.tsx
```

4.1 AFTER-ACTION DASHBOARD

tsx

None

```tsx
// components/review/dashboards/AfterActionDashboard.tsx
import React, { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';

interface AfterActionDashboardProps {
  eventId: string;
  venueId: string;
}
```

```
export const AfterActionDashboard:
React.FC<AfterActionDashboardProps> = ({
  eventId,
  venueId
}) => {
  const [report, setReport] = useState<AfterActionReport |
null>(null);
  const [metrics, setMetrics] = useState<EventPerformanceMetrics
| null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    loadAfterActionData();
  }, [eventId]);

  const loadAfterActionData = async () => {
    try {
      const [reportData, metricsData] = await Promise.all([
        fetchReport(eventId),
        fetchMetrics(eventId)
      ]);
      setReport(reportData);
      setMetrics(metricsData);
    } catch (error) {
      console.error('Failed to load after-action data:', error);
    } finally {
      setLoading(false);
    }
  };

  if (loading) return <LoadingSpinner />;
  if (!report || !metrics) return <ErrorState />;

  return (
    <div className="after-action-dashboard">
      {/* Header Section - Navy background */}
```

```
      <div className="dashboard-header bg-primary-navy text-white
p-6">
        <div className="flex justify-between items-start">
          <div>
            <h1 className="text-3xl font-bold mb-2">After-Action
Review</h1>
            <p className="text-lg opacity-90">
              {report.event_date} • {venueId}
            </p>
          </div>
          <div className="flex gap-3">
            <button className="btn-secondary-teal">
              <DownloadIcon /> Export Report
            </button>
            <button className="btn-accent-green">
              <ShareIcon /> Share
            </button>
          </div>
        </div>

        {/* Quick Stats - Accent Green highlights */}
        <div className="grid grid-cols-4 gap-4 mt-6">
          <QuickStatCard
            label="Overall Score"
            value={report.executive_summary?.overall_rating || 0}
            max={10}
            color="accent-green"
            trend={metrics.historical_comparison?.variance_pct}
          />
          <QuickStatCard
            label="Incidents Prevented"
            value={metrics.incidents_prevented}
            icon={<ShieldIcon />}
            color="accent-green"
          />
          <QuickStatCard
```

```
            label="Response Time"
value={`${Math.round(metrics.avg_response_time_seconds / 60)}m`}
            improvement="70%"
            color="secondary-teal"
          />
          <QuickStatCard
            label="Guest Satisfaction"
            value={metrics.guest_satisfaction_score.toFixed(1)}
            max={5}
            icon={<StarIcon />}
            color="accent-green"
          />
        </div>
      </div>

      {/* Main Content */}
      <div className="dashboard-content p-6 bg-light-bg">
        <div className="grid grid-cols-3 gap-6">
          {/* Left Column - Executive Summary */}
          <div className="col-span-2 space-y-6">
            <ExecutiveSummaryCard
summary={report.executive_summary} />
            <PerformanceMetricsSection metrics={metrics} />
            <IncidentAnalysisSection
incidents={report.incident_analysis} />
            <RecommendationsSection
              recommendations={report.recommendations}
              onRecommendationClick={handleRecommendationClick}
            />
          </div>

          {/* Right Column - Insights & Actions */}
          <div className="space-y-6">
            <KeyAchievementsPanel
achievements={report.executive_summary?.key_achievements} />
```

```
            <CriticalIssuesPanel
issues={report.executive_summary?.critical_issues} />
            <ComplianceStatusCard
compliance={report.compliance_summary} />
            <BenchmarkComparisonCard
              venue={metrics.venue_id}
              league={metrics.league_avg_comparison}
            />
          </div>
        </div>
      </div>
    </div>
  );
};

// Supporting Components

const QuickStatCard: React.FC<{
  label: string;
  value: number | string;
  max?: number;
  icon?: React.ReactNode;
  color?: string;
  trend?: number;
  improvement?: string;
}> = ({ label, value, max, icon, color = 'accent-green', trend,
improvement }) => {
  const colorClasses = {
    'accent-green': 'text-accent-green border-accent-green',
    'secondary-teal': 'text-secondary-teal
border-secondary-teal',
    'primary-navy': 'text-primary-navy border-primary-navy'
  };

  return (
```

```
    <div className="quick-stat-card bg-white rounded-lg p-4
border-l-4"
         style={{ borderLeftColor: `var(--${color})` }}>
      <div className="flex items-center justify-between mb-2">
        <span className="text-body-gray text-sm">{label}</span>
        {icon && <span
className={colorClasses[color]}>{icon}</span>}
      </div>
      <div className="flex items-baseline">
        <span className={`text-3xl font-bold
${colorClasses[color]}`}>
          {value}
        </span>
        {max && <span className="text-body-gray ml-1">/
{max}</span>}
      </div>
      {(trend || improvement) && (
        <div className="flex items-center mt-2 text-sm">
          {trend && (
            <span className={trend > 0 ? 'text-accent-green' :
'text-red-500'}>
              <TrendIcon direction={trend > 0 ? 'up' : 'down'} />
              {Math.abs(trend)}%
            </span>
          )}
          {improvement && (
            <span className="text-accent-green ml-2">
              ↑ {improvement} improvement
            </span>
          )}
        </div>
      )}
    </div>
  );
};
```

```
const ExecutiveSummaryCard: React.FC<{ summary: any }> = ({
summary }) => {
  return (
    <div className="card bg-white rounded-lg shadow p-6">
      <h2 className="text-2xl font-bold text-primary-navy mb-4">
        Executive Summary
      </h2>
      <div className="prose text-body-gray">
        <p className="text-lg leading-relaxed">
          {summary?.event_overview}
        </p>
      </div>
      <div className="mt-6">
        <p className="text-body-gray
mb-2">{summary?.recommendations_summary}</p>
      </div>
    </div>
  );
};

const PerformanceMetricsSection: React.FC<{ metrics:
EventPerformanceMetrics }> = ({
  metrics
}) => {
  const categories = [
    {
      name: 'Safety & Security',
      score: calculateSafetyScore(metrics),
      color: 'accent-green',
      metrics: [
        { label: 'Emergency Activations', value:
metrics.emergency_activations },
        { label: 'Response Time', value:
`${Math.round(metrics.avg_response_time_seconds / 60)}m` },
        { label: 'Incidents Prevented', value:
metrics.incidents_prevented }
```

```
      ]
    },
    {
      name: 'Operational Excellence',
      score: calculateOperationalScore(metrics),
      color: 'secondary-teal',
      metrics: [
        { label: 'System Uptime', value:
`${metrics.system_uptime_pct}%` },
        { label: 'Efficiency', value:
`${metrics.resource_efficiency_pct}%` },
        { label: 'Integration Failures', value:
metrics.integration_failures }
      ]
    },
    {
      name: 'Guest Experience',
      score: (metrics.guest_satisfaction_score / 5) * 100,
      color: 'accent-green',
      metrics: [
        { label: 'Satisfaction', value:
metrics.guest_satisfaction_score.toFixed(1) },
        { label: 'NPS Score', value: metrics.nps_score },
        { label: 'Complaints', value: metrics.complaint_count }
      ]
    },
    {
      name: 'Predictive Accuracy',
      score: metrics.prediction_accuracy_pct,
      color: 'secondary-teal',
      metrics: [
        { label: 'Predictions Made', value:
metrics.predictions_made },
        { label: 'Accurate', value: metrics.predictions_accurate
},
```

```
        { label: 'False Positives', value:
metrics.false_positives }
      ]
    }
  ];

  return (
    <div className="card bg-white rounded-lg shadow p-6">
      <h2 className="text-2xl font-bold text-primary-navy mb-6">
        Performance Metrics
      </h2>
      <div className="grid grid-cols-2 gap-6">
        {categories.map((category, index) => (
          <PerformanceCategoryCard key={index} {...category} />
        ))}
      </div>
    </div>
  );
};

const PerformanceCategoryCard: React.FC<{
  name: string;
  score: number;
  color: string;
  metrics: { label: string; value: number | string }[];
}> = ({ name, score, color, metrics }) => {
  return (
    <div className="performance-category border
border-light-border rounded-lg p-4">
      <div className="flex items-center justify-between mb-4">
        <h3 className="text-lg font-semibold
text-primary-navy">{name}</h3>
        <span className={`text-2xl font-bold text-${color}`}>
          {Math.round(score)}
        </span>
      </div>
```

```
      {/* Progress Bar */}
      <div className="w-full bg-light-bg rounded-full h-2 mb-4">
        <div
          className={`bg-${color} h-2 rounded-full transition-all
duration-500`}
          style={{ width: `${score}%` }}
        />
      </div>

      {/* Metrics */}
      <div className="space-y-2">
        {metrics.map((metric, idx) => (
          <div key={idx} className="flex justify-between
text-sm">
            <span
className="text-body-gray">{metric.label}</span>
            <span className="font-semibold
text-primary-navy">{metric.value}</span>
          </div>
        ))}
      </div>
    </div>
  );
};
```

4.2 INCIDENT ANALYSIS COMPONENT

tsx

```
None
// components/review/incidents/IncidentAnalysis.tsx
import React, { useState } from 'react';

interface IncidentAnalysisProps {
```

```
  incidentId: string;
  eventId: string;
}

export const IncidentAnalysis: React.FC<IncidentAnalysisProps> =
({
  incidentId,
  eventId
}) => {
  const [analysis, setAnalysis] = useState<IncidentAnalysis |
null>(null);
  const [activeTab, setActiveTab] = useState<'timeline' |
'root_cause' | 'response' | 'lessons'>('timeline');

  return (
    <div className="incident-analysis bg-white rounded-lg
shadow">
      {/* Header with Status Badge */}
      <div className="border-b border-light-border p-6">
        <div className="flex items-center justify-between">
          <div>
            <h2 className="text-2xl font-bold text-primary-navy
mb-2">
              Incident Analysis
            </h2>
            <div className="flex items-center gap-3">
              <SeverityBadge level={analysis?.severity_level} />
              <PreventabilityBadge
type={analysis?.preventability} />
              <span className="text-body-gray text-sm">
                Analysis completed
{formatDate(analysis?.analysis_date)}
              </span>
            </div>
          </div>
          <div className="flex gap-2">
```

```
            <button className="btn-secondary-teal">
              <ExportIcon /> Export
            </button>
            <button className="btn-accent-green">
              <RecommendIcon /> Generate Recommendations
            </button>
          </div>
        </div>
      </div>

      {/* Tab Navigation - Secondary Teal */}
      <div className="border-b border-light-border">
        <div className="flex gap-1 px-6">
          {['timeline', 'root_cause', 'response',
'lessons'].map(tab => (
            <button
              key={tab}
              onClick={() => setActiveTab(tab as any)}
              className={`px-6 py-3 font-semibold
transition-colors ${
                activeTab === tab
                  ? 'text-secondary-teal border-b-2
border-secondary-teal'
                  : 'text-body-gray hover:text-primary-navy'
              }`}
            >
              {formatTabName(tab)}
            </button>
          ))}
        </div>
      </div>

      {/* Tab Content */}
      <div className="p-6">
        {activeTab === 'timeline' && <TimelineView
analysis={analysis} />}
```

```
        {activeTab === 'root_cause' && <RootCauseView
analysis={analysis} />}
        {activeTab === 'response' && <ResponseAnalysisView
analysis={analysis} />}
        {activeTab === 'lessons' && <LessonsLearnedView
analysis={analysis} />}
      </div>
    </div>
  );
};

const TimelineView: React.FC<{ analysis: IncidentAnalysis }> = ({
analysis }) => {
  const timelineEvents = [
    {
      time: 0,
      label: 'Incident Detected',
      method: analysis.detection_method,
      color: 'accent-green'
    },
    {
      time: analysis.detection_time_seconds,
      label: 'Detection Confirmed',
      details: 'Automated validation completed',
      color: 'secondary-teal'
    },
    {
      time: analysis.response_time_seconds,
      label: 'Response Initiated',
      details: 'Multi-agency coordination activated',
      color: 'secondary-teal'
    },
    {
      time: analysis.resolution_time_seconds,
      label: 'Incident Resolved',
      effectiveness: analysis.response_effectiveness,
```

```
      color: 'accent-green'
    }
  ];

  return (
    <div className="timeline-view">
      {/* Visual Timeline */}
      <div className="relative">
        {timelineEvents.map((event, index) => (
          <TimelineEvent key={index} event={event} isLast={index
=== timelineEvents.length - 1} />
        ))}
      </div>

      {/* Response Metrics */}
      <div className="grid grid-cols-3 gap-4 mt-8">
        <MetricCard
          label="Detection Time"
          value={formatSeconds(analysis.detection_time_seconds)}
          icon={<ClockIcon />}
          color="secondary-teal"
        />
        <MetricCard
          label="Response Time"
          value={formatSeconds(analysis.response_time_seconds)}
          icon={<ResponseIcon />}
          color="secondary-teal"
        />
        <MetricCard
          label="Resolution Time"
          value={formatSeconds(analysis.resolution_time_seconds)}
          icon={<CheckIcon />}
          color="accent-green"
        />
      </div>
    </div>
```

```
  );
};

const RootCauseView: React.FC<{ analysis: IncidentAnalysis }> =
({ analysis }) => {
  return (
    <div className="root-cause-view space-y-6">
      {/* Root Cause Card */}
      {analysis.root_cause_identified && (
        <div className="bg-light-bg border-l-4
border-accent-green rounded-lg p-6">
          <div className="flex items-start gap-3">
            <div className="p-2 bg-accent-green rounded-full">
              <CheckCircleIcon className="text-white" />
            </div>
            <div>
              <h3 className="text-lg font-bold text-primary-navy
mb-2">
                Root Cause Identified
              </h3>
              <p className="text-body-gray leading-relaxed">
                {analysis.root_cause_description}
              </p>
            </div>
          </div>
        </div>
      )}

      {/* Contributing Factors */}
      <div className="card border border-light-border rounded-lg
p-6">
        <h3 className="text-lg font-bold text-primary-navy mb-4">
          Contributing Factors
        </h3>
        <ul className="space-y-3">
          {analysis.contributing_factors.map((factor, index) => (
```

```
            <li key={index} className="flex items-start gap-3">
              <span className="text-secondary-teal">•</span>
              <span className="text-body-gray">{factor}</span>
            </li>
          ))}
        </ul>
      </div>

      {/* Prevention Opportunities */}
      {analysis.could_have_been_prevented && (
        <div className="card border border-light-border
rounded-lg p-6">
          <h3 className="text-lg font-bold text-primary-navy
mb-4">
            Prevention Opportunities
          </h3>
          <div className="space-y-4">
            {analysis.prevention_opportunities.map((opportunity,
index) => (
              <div key={index} className="flex items-start gap-3
p-3 bg-light-bg rounded">
                <LightbulbIcon className="text-accent-green
flex-shrink-0 mt-1" />
                <span
className="text-body-gray">{opportunity}</span>
              </div>
            ))}
          </div>
        </div>
      )}

      {/* Similar Historical Incidents */}
      {analysis.similar_incidents_historical > 0 && (
        <div className="card border border-light-border
rounded-lg p-6">
```

```
          <div className="flex items-center justify-between
mb-4">
            <h3 className="text-lg font-bold text-primary-navy">
              Historical Pattern Detected
            </h3>
            <span className="px-3 py-1 bg-secondary-teal
text-white rounded-full text-sm font-semibold">
              {analysis.similar_incidents_historical} similar
incidents
            </span>
          </div>
          <p className="text-body-gray mb-4">
            This type of incident has occurred
{analysis.similar_incidents_historical} times previously.
            Review recommended to identify systemic improvements.
          </p>
          <button className="btn-secondary-teal w-full">
            View Similar Incidents
          </button>
        </div>
      )}
    </div>
  );
};

const ResponseAnalysisView: React.FC<{ analysis: IncidentAnalysis
}> = ({ analysis }) => {
  const effectivenessColors = {
    'excellent': 'accent-green',
    'good': 'secondary-teal',
    'adequate': 'yellow-500',
    'poor': 'red-500'
  };

  return (
    <div className="response-analysis-view space-y-6">
```

```
      {/* Effectiveness Rating */}
      <div className="card bg-light-bg rounded-lg p-6">
        <div className="flex items-center justify-between mb-4">
          <h3 className="text-lg font-bold text-primary-navy">
            Response Effectiveness
          </h3>
          <span className={`px-4 py-2
bg-${effectivenessColors[analysis.response_effectiveness]}
text-white rounded-lg text-lg font-bold capitalize`}>
            {analysis.response_effectiveness}
          </span>
        </div>

        {/* Response Metrics Grid */}
        <div className="grid grid-cols-2 gap-4 mt-4">
          <div className="text-center p-4 bg-white rounded">
            <div className="text-3xl font-bold
text-secondary-teal mb-1">
              {analysis.people_impacted}
            </div>
            <div className="text-sm text-body-gray">People
Impacted</div>
          </div>
          <div className="text-center p-4 bg-white rounded">
            <div className="text-3xl font-bold
text-secondary-teal mb-1">
              {analysis.areas_affected.length}
            </div>
            <div className="text-sm text-body-gray">Areas
Affected</div>
          </div>
        </div>
      </div>

      {/* Impact Assessment */}
```

```
      <div className="card border border-light-border rounded-lg
p-6">
        <h3 className="text-lg font-bold text-primary-navy mb-4">
          Impact Assessment
        </h3>
        <div className="space-y-3">
          <ImpactRow
            label="Financial Impact"
            value={formatCurrency(analysis.financial_impact)}
            severity="medium"
          />
          <ImpactRow
            label="Reputational Impact"
            value={analysis.reputational_impact}

severity={getReputationSeverity(analysis.reputational_impact)}
          />
          <ImpactRow
            label="Operational Impact"
            value={`${analysis.areas_affected.length} areas
affected`}
            severity="low"
          />
        </div>
      </div>

      {/* Response Gaps */}
      {analysis.response_gaps.length > 0 && (
        <div className="card border-l-4 border-yellow-500
bg-yellow-50 rounded-lg p-6">
          <h3 className="text-lg font-bold text-primary-navy mb-4
flex items-center gap-2">
            <AlertIcon className="text-yellow-500" />
            Response Gaps Identified
          </h3>
          <ul className="space-y-2">
```

```
            {analysis.response_gaps.map((gap, index) => (
              <li key={index} className="flex items-start gap-2">
                <span className="text-yellow-500
font-bold">→</span>
                <span className="text-body-gray">{gap}</span>
              </li>
            ))}
          </ul>
        </div>
      )}
    </div>
  );
};

const LessonsLearnedView: React.FC<{ analysis: IncidentAnalysis
}> = ({ analysis }) => {
  return (
    <div className="lessons-learned-view space-y-6">
      {/* Lessons Learned */}
      <div className="card border border-light-border rounded-lg
p-6">
        <h3 className="text-lg font-bold text-primary-navy mb-4
flex items-center gap-2">
          <BookIcon className="text-secondary-teal" />
          Lessons Learned
        </h3>
        <div className="space-y-4">
          {analysis.lessons_learned.map((lesson, index) => (
            <div key={index} className="p-4 bg-light-bg
rounded-lg">
              <div className="flex items-start gap-3">
                <span className="flex-shrink-0 w-8 h-8
bg-secondary-teal text-white rounded-full flex items-center
justify-center font-bold">
                  {index + 1}
                </span>
```

```
              <p className="text-body-gray pt-1">{lesson}</p>
            </div>
          </div>
        ))}
      </div>
    </div>

    {/* Best Practices Identified */}
    {analysis.best_practices_identified.length > 0 && (
      <div className="card border-l-4 border-accent-green
bg-white rounded-lg p-6">
        <h3 className="text-lg font-bold text-primary-navy mb-4
flex items-center gap-2">
          <StarIcon className="text-accent-green" />
          Best Practices Identified
        </h3>
        <div className="space-y-3">
          {analysis.best_practices_identified.map((practice,
index) => (
            <div key={index} className="flex items-start gap-3
p-3 bg-light-bg rounded">
              <CheckIcon className="text-accent-green
flex-shrink-0 mt-1" />
              <span
className="text-body-gray">{practice}</span>
            </div>
          ))}
        </div>
      </div>
    )}

    {/* Improvement Recommendations */}
    <div className="card border border-light-border rounded-lg
p-6">
      <h3 className="text-lg font-bold text-primary-navy mb-4">
        Recommended Improvements
```

```
        </h3>
        <div className="space-y-3">
          {analysis.improvements_recommended.map((improvement,
index) => (
            <div key={index} className="flex items-center
justify-between p-4 bg-light-bg rounded-lg
hover:bg-secondary-teal hover:bg-opacity-10 transition-colors
cursor-pointer">
              <div className="flex items-start gap-3 flex-1">
                <ArrowRightIcon className="text-accent-green
flex-shrink-0 mt-1" />
                <span
className="text-body-gray">{improvement}</span>
              </div>
              <button className="btn-sm btn-accent-green">
                Create Recommendation
              </button>
            </div>
          ))}
        </div>
      </div>

      {/* Evidence & Documentation */}
      <div className="card border border-light-border rounded-lg
p-6">
        <h3 className="text-lg font-bold text-primary-navy mb-4">
          Evidence & Documentation
        </h3>
        <div className="grid grid-cols-2 gap-4">
          <EvidenceSection
            title="Evidence Ledger"
            count={analysis.evidence_ledger_refs.length}
            icon={<BlockchainIcon />}
            color="accent-green"
          />
          <EvidenceSection
```

```
            title="Video Evidence"
            count={analysis.video_evidence_refs.length}
            icon={<VideoIcon />}
            color="secondary-teal"
          />
          <EvidenceSection
            title="Documentation"
            count={analysis.documentation_refs.length}
            icon={<DocumentIcon />}
            color="secondary-teal"
          />
          <EvidenceSection
            title="Witness Statements"
            count={analysis.witness_statements.length}
            icon={<UserIcon />}
            color="accent-green"
          />
        </div>
      </div>
    </div>
  );
};
```

4.3 IMPROVEMENT RECOMMENDATIONS COMPONENT

tsx

None

```
// components/review/improvements/RecommendationList.tsx
import React, { useState, useEffect } from 'react';

interface RecommendationListProps {
  venueId: string;
  filters?: {
    status?: string;
```

```
    priority?: string;
    type?: string;
  };
}

export const RecommendationList:
React.FC<RecommendationListProps> = ({
  venueId,
  filters
}) => {
  const [recommendations, setRecommendations] =
useState<ImprovementRecommendation[]>([]);
  const [view, setView] = useState<'list' | 'board' |
'timeline'>('board');
  const [selectedRec, setSelectedRec] = useState<string |
null>(null);

  const priorityConfig = {
    critical: { color: 'red-500', bg: 'red-50', icon:
<AlertCircleIcon /> },
    high: { color: 'yellow-500', bg: 'yellow-50', icon:
<AlertIcon /> },
    medium: { color: 'secondary-teal', bg: 'blue-50', icon:
<InfoIcon /> },
    low: { color: 'body-gray', bg: 'gray-50', icon: <InfoIcon />
}
  };

  return (
    <div className="recommendation-list">
      {/* Header with Filters and View Toggle */}
      <div className="bg-white border-b border-light-border p-6">
        <div className="flex items-center justify-between mb-6">
          <h2 className="text-2xl font-bold text-primary-navy">
            Improvement Recommendations
          </h2>
```

```
          <button className="btn-accent-green">
            <PlusIcon /> New Recommendation
          </button>
        </div>

        {/* Summary Stats */}
        <div className="grid grid-cols-4 gap-4 mb-6">
          <StatCard label="Total" value={recommendations.length}
color="primary-navy" />
          <StatCard
            label="In Progress"
            value={recommendations.filter(r =>
r.implementation_status === 'in_progress').length}
            color="secondary-teal"
          />
          <StatCard
            label="Completed"
            value={recommendations.filter(r =>
r.implementation_status === 'completed').length}
            color="accent-green"
          />
          <StatCard
            label="Pending Approval"
            value={recommendations.filter(r =>
r.implementation_status === 'proposed').length}
            color="yellow-500"
          />
        </div>

        {/* Filters and View Toggle */}
        <div className="flex items-center justify-between">
          <div className="flex gap-3">
            <FilterDropdown label="Status"
options={statusOptions} />
            <FilterDropdown label="Priority"
options={priorityOptions} />
```

```
            <FilterDropdown label="Type" options={typeOptions} />
          </div>
          <ViewToggle view={view} onChange={setView} />
        </div>
      </div>

      {/* Content Area */}
      <div className="p-6 bg-light-bg">
        {view === 'board' && <KanbanBoard
recommendations={recommendations} />}
        {view === 'list' && <ListView
recommendations={recommendations} />}
        {view === 'timeline' && <TimelineView
recommendations={recommendations} />}
      </div>

      {/* Recommendation Detail Drawer */}
      {selectedRec && (
        <RecommendationDrawer
          recommendationId={selectedRec}
          onClose={() => setSelectedRec(null)}
        />
      )}
    </div>
  );
};

const KanbanBoard: React.FC<{ recommendations:
ImprovementRecommendation[] }> = ({
  recommendations
}) => {
  const columns = [
    { status: 'proposed', title: 'Proposed', color: 'body-gray'
},
    { status: 'approved', title: 'Approved', color:
'accent-green' },
```

```
    { status: 'in_progress', title: 'In Progress', color:
'secondary-teal' },
    { status: 'completed', title: 'Completed', color:
'accent-green' }
  ];

  return (
    <div className="kanban-board flex gap-4 overflow-x-auto
pb-4">
      {columns.map(column => (
        <KanbanColumn
          key={column.status}
          title={column.title}
          color={column.color}
          recommendations={recommendations.filter(r =>
r.implementation_status === column.status)}
        />
      ))}
    </div>
  );
};

const KanbanColumn: React.FC<{
  title: string;
  color: string;
  recommendations: ImprovementRecommendation[];
}> = ({ title, color, recommendations }) => {
  return (
    <div className="kanban-column flex-shrink-0 w-80">
      {/* Column Header */}
      <div className={`bg-${color} bg-opacity-10 rounded-t-lg
p-4`}>
        <div className="flex items-center justify-between">
          <h3 className={`font-bold text-${color}`}>{title}</h3>
          <span className={`px-2 py-1 bg-${color} text-white
rounded-full text-sm font-semibold`}>
```

```
            {recommendations.length}
          </span>
        </div>
      </div>

      {/* Column Content */}
      <div className="space-y-3 p-3 bg-white border
border-light-border border-t-0 rounded-b-lg min-h-96">
        {recommendations.map(rec => (
          <RecommendationCard key={rec.recommendation_id}
recommendation={rec} />
        ))}
      </div>
    </div>
  );
};

const RecommendationCard: React.FC<{ recommendation:
ImprovementRecommendation }> = ({
  recommendation: rec
}) => {
  const priorityColors = {
    critical: 'red-500',
    high: 'yellow-500',
    medium: 'secondary-teal',
    low: 'body-gray'
  };

  return (
    <div className="recommendation-card bg-white border
border-light-border rounded-lg p-4 hover:shadow-md
transition-shadow cursor-pointer">
      {/* Header */}
      <div className="flex items-start justify-between mb-3">
        <h4 className="font-semibold text-primary-navy text-sm
line-clamp-2">
```

```
          {rec.recommendation_title}
        </h4>
        <PriorityBadge priority={rec.priority_level} />
      </div>

      {/* Type Tag */}
      <div className="mb-3">
        <span className="inline-block px-2 py-1 bg-light-bg
text-body-gray text-xs rounded">
          {formatType(rec.recommendation_type)}
        </span>
      </div>

      {/* Description Preview */}
      <p className="text-body-gray text-sm line-clamp-2 mb-3">
        {rec.recommendation_description}
      </p>

      {/* Footer Metrics */}
      <div className="flex items-center justify-between text-xs
text-body-gray pt-3 border-t border-light-border">
        <div className="flex items-center gap-3">
          <span className="flex items-center gap-1">
            <DollarIcon className="w-3 h-3" />
            {formatCurrency(rec.estimated_cost)}
          </span>
          <span className="flex items-center gap-1">
            <ClockIcon className="w-3 h-3" />
            {rec.estimated_hours}h
          </span>
        </div>
        {rec.expected_roi && (
          <span className="text-accent-green font-semibold">
            ROI: {rec.expected_roi}%
          </span>
        )}
```

```
      </div>

      {/* Progress Bar (if in progress) */}
      {rec.implementation_status === 'in_progress' && (
        <div className="mt-3">
          <div className="w-full bg-light-bg rounded-full h-1.5">
            <div
              className="bg-secondary-teal h-1.5 rounded-full
transition-all"
              style={{ width:
`${getImplementationProgress(rec)}%` }}
            />
          </div>
        </div>
      )}

      {/* Target Date (if set) */}
      {rec.target_completion_date && (
        <div className="mt-2 text-xs text-body-gray flex
items-center gap-1">
          <CalendarIcon className="w-3 h-3" />
          Due: {formatDate(rec.target_completion_date)}
        </div>
      )}
    </div>
  );
};
```

4.4 PERFORMANCE DASHBOARD

typescript

TypeScript

```
// components/review/dashboards/PerformanceDashboard.tsx
import React, { useState, useEffect } from 'react';
```

```
import { Line, Bar, Radar } from 'react-chartjs-2';

interface PerformanceDashboardProps {
  venueId: string;
  period?: 'current_month' | 'last_30_days' | 'current_quarter' |
'custom';
  startDate?: Date;
  endDate?: Date;
}

export const PerformanceDashboard:
React.FC<PerformanceDashboardProps> = ({
  venueId,
  period = 'current_month',
  startDate,
  endDate
}) => {
  const [dashboard, setDashboard] = useState<any>(null);
  const [comparison, setComparison] = useState<'historical' |
'league' | 'peer'>('league');
  const [loading, setLoading] = useState(true);

  const chartColors = {
    primary: '#1B3A4B',
    secondary: '#3A7B8A',
    accent: '#9AAF45',
    light: '#F4F6F7'
  };

  return (
    <div className="performance-dashboard">
      {/* Header with Period Selector */}
      <div className="bg-primary-navy text-white p-6
rounded-t-lg">
        <div className="flex items-center justify-between">
          <div>
```

```
            <h1 className="text-3xl font-bold mb-2">Performance
Dashboard</h1>
            <p className="opacity-90">{formatPeriod(period,
startDate, endDate)}</p>
          </div>
          <PeriodSelector period={period}
onChange={handlePeriodChange} />
        </div>

        {/* Overall Score - Large Display */}
        <div className="mt-6 text-center">
          <div className="text-6xl font-bold text-accent-green
mb-2">
            {dashboard?.summary?.overall_score || 0}
          </div>
          <div className="text-xl opacity-90">Overall Performance
Score</div>
        </div>
      </div>

      {/* Quick Metrics Bar */}
      <div className="grid grid-cols-5 gap-4 p-6 bg-white
border-b">
        <QuickMetric
          label="Total Events"
          value={dashboard?.summary?.total_events}
          icon={<CalendarIcon />}
        />
        <QuickMetric
          label="Incidents Prevented"
          value={dashboard?.summary?.incidents_prevented}

trend={calculateTrend(dashboard?.trends?.incidents_prevented)}
          icon={<ShieldCheckIcon />}
          color="accent-green"
        />
```

```
        <QuickMetric
          label="Avg Response Time"

value={`${Math.round(dashboard?.summary?.avg_response_time_second
s / 60)}m`}

trend={calculateTrend(dashboard?.trends?.avg_response_time_second
s, true)} // Lower is better
          icon={<BoltIcon />}
          color="secondary-teal"
        />
        <QuickMetric
          label="Guest Satisfaction"

value={dashboard?.summary?.guest_satisfaction_avg?.toFixed(1)}
          max={5}

trend={calculateTrend(dashboard?.trends?.guest_satisfaction_avg)}
          icon={<StarIcon />}
          color="accent-green"
        />
        <QuickMetric
          label="Compliance Score"
          value={`${dashboard?.summary?.compliance_score}%`}

trend={calculateTrend(dashboard?.trends?.compliance_score)}
          icon={<CheckCircleIcon />}
          color="accent-green"
        />
      </div>

      {/* Main Content Grid */}
      <div className="p-6 bg-light-bg grid grid-cols-3 gap-6">
        {/* Left Column - Charts */}
        <div className="col-span-2 space-y-6">
          {/* Performance Trends Chart */}
```

```
          <div className="card bg-white rounded-lg shadow p-6">
            <div className="flex items-center justify-between
mb-6">
              <h3 className="text-xl font-bold
text-primary-navy">
                Performance Trends
              </h3>
              <ComparisonToggle value={comparison}
onChange={setComparison} />
            </div>
            <PerformanceTrendsChart
              data={dashboard?.trends}
              comparison={comparison}
              colors={chartColors}
            />
          </div>

          {/* Category Performance Radar */}
          <div className="card bg-white rounded-lg shadow p-6">
            <h3 className="text-xl font-bold text-primary-navy
mb-6">
              Category Performance
            </h3>
            <CategoryRadarChart
              scores={dashboard?.category_scores}
              colors={chartColors}
            />
          </div>

          {/* Metric Details Table */}
          <div className="card bg-white rounded-lg shadow p-6">
            <h3 className="text-xl font-bold text-primary-navy
mb-6">
              Detailed Metrics
            </h3>
            <MetricsTable
```

```
              metrics={dashboard?.trends}
              comparison={comparison}
            />
          </div>
        </div>

        {/* Right Column - Insights */}
        <div className="space-y-6">
          {/* Alerts & Notifications */}
          <div className="card bg-white rounded-lg shadow p-6">
            <h3 className="text-lg font-bold text-primary-navy
mb-4">
              Performance Alerts
            </h3>
            <div className="space-y-3">
              {dashboard?.alerts?.declining_metrics?.map((metric,
index) => (
                <AlertCard
                  key={index}
                  type="warning"
                  title={formatMetricName(metric)}
                  message="Performance declining - review
recommended"
                  color="yellow-500"
                />
              ))}

{dashboard?.alerts?.notable_achievements?.map((achievement,
index) => (
                <AlertCard
                  key={index}
                  type="success"
                  title="Achievement"
                  message={achievement}
                  color="accent-green"
                />
```

```
            ))}
          </div>
        </div>

        {/* Improvement Opportunities */}
        <div className="card bg-white rounded-lg shadow p-6">
          <h3 className="text-lg font-bold text-primary-navy
mb-4 flex items-center gap-2">
            <LightbulbIcon className="text-accent-green" />
            Improvement Opportunities
          </h3>
          <div className="space-y-3">

{dashboard?.alerts?.improvement_opportunities?.map((opp, index)
=> (
              <div key={index} className="p-3 bg-light-bg
rounded hover:bg-secondary-teal hover:bg-opacity-10
cursor-pointer transition-colors">
                <p className="text-sm text-body-gray">{opp}</p>
              </div>
            ))}
          </div>
        </div>

        {/* Top Performers */}
        <div className="card bg-white rounded-lg shadow p-6">
          <h3 className="text-lg font-bold text-primary-navy
mb-4">
            Top Performing Metrics
          </h3>
          <div className="space-y-3">
            {getTopMetrics(dashboard?.trends, 3).map((metric,
index) => (
              <div key={index} className="flex items-center
justify-between p-3 bg-accent-green bg-opacity-10 rounded">
```

```
                  <span className="text-sm font-semibold
text-primary-navy">
                    {formatMetricName(metric.name)}
                  </span>
                  <span className="text-accent-green font-bold">
                    {metric.score}
                  </span>
                </div>
              ))}
            </div>
          </div>

          {/* Quick Actions */}
          <div className="card bg-white rounded-lg shadow p-6">
            <h3 className="text-lg font-bold text-primary-navy
mb-4">
              Quick Actions
            </h3>
            <div className="space-y-2">
              <button className="btn-secondary-teal w-full
justify-center">
                <ReportIcon /> Generate Report
              </button>
              <button className="btn-secondary-teal w-full
justify-center">
                <CompareIcon /> Compare with League
              </button>
              <button className="btn-accent-green w-full
justify-center">
                <DownloadIcon /> Export Data
              </button>
            </div>
          </div>
        </div>
      </div>
    </div>
```

```
  );
};

// Chart Component Examples

const PerformanceTrendsChart: React.FC<{
  data: any;
  comparison: string;
  colors: any;
}> = ({ data, comparison, colors }) => {
  // Transform data for chart
  const chartData = {
    labels: data?.dates || [],
    datasets: [
      {
        label: 'Your Performance',
        data: data?.overall_scores || [],
        borderColor: colors.accent,
        backgroundColor: `${colors.accent}20`,
        fill: true,
        tension: 0.4
      },
      ...(comparison === 'league' ? [{
        label: 'League Average',
        data: data?.league_average || [],
        borderColor: colors.secondary,
        backgroundColor: 'transparent',
        borderDash: [5, 5]
      }] : [])
    ]
  };

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    plugins: {
```

```
      legend: {
        position: 'top' as const,
      },
      tooltip: {
        mode: 'index' as const,
        intersect: false,
      }
    },
    scales: {
      y: {
        beginAtZero: true,
        max: 100,
        grid: {
          color: colors.light
        }
      },
      x: {
        grid: {
          display: false
        }
      }
    }
  };

  return (
    <div style={{ height: '300px' }}>
      <Line data={chartData} options={options} />
    </div>
  );
};

const CategoryRadarChart: React.FC<{
  scores: any;
  colors: any;
}> = ({ scores, colors }) => {
  const chartData = {
```

```
    labels: ['Safety', 'Operations', 'Efficiency',
'Satisfaction'],
    datasets: [{
      label: 'Performance',
      data: [
        scores?.safety || 0,
        scores?.operations || 0,
        scores?.efficiency || 0,
        scores?.satisfaction || 0
      ],
      backgroundColor: `${colors.accent}30`,
      borderColor: colors.accent,
      borderWidth: 2,
      pointBackgroundColor: colors.accent,
      pointBorderColor: '#fff',
      pointHoverBackgroundColor: '#fff',
      pointHoverBorderColor: colors.accent
    }]
  };

  const options = {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
      r: {
        angleLines: {
          color: colors.light
        },
        grid: {
          color: colors.light
        },
        pointLabels: {
          font: {
            size: 14,
            weight: 'bold' as const
          },
```

```
          color: colors.primary
        },
        ticks: {
          display: false
        },
        suggestedMin: 0,
        suggestedMax: 100
      }
    },
    plugins: {
      legend: {
        display: false
      }
    }
  };

  return (
    <div style={{ height: '400px' }}>
      <Radar data={chartData} options={options} />
    </div>
  );
};
```

[APPROACHING TOKEN LIMIT]

CONTINUATION PROMPT FOR NEXT THREAD: